MyAPDLCall.py

```
# MyAPDLCall.py
# When called, this document runs Eigenbuckling Analysis and Nonlinear and Returns mass

# -> INPUT:
#       [SWcoor]    -> Coordinates from Solidworks IGES
#       [var]       -> Radii variables
#       [Misc]      -> Miscellaneous Data (force, mesh etc.)
# -> OUTPUT:
#       [Mass]      -> Mass of total assembly

# Pseudo code
#   Import [SWcoor], [var] and [Misc]
#   Remove content from "AnsoutEigen" and "AnsoutNonlin" folders
#   Call APDL_Eigen to create .txt input file
#       1. Run Analysis via os.system
#       2. Read first positive eigenvalue
#       3. Calculate Imperfection Force
#   Call APDL_Nonlin to create .txt input file
#       1. Add Imperfection force as input
#       2. Run Analysis via os.system
#       3. Read Mass of assembly
#   Return Mass

# Import packages

import os
import time
import numpy as np

# Import Functions
from APDL_Input import InputFun


def RunAPDL(mapdl,var,Misc,opti_settings):
    # Handle List or Misc
    if isinstance(var, dict):
        var_dict = var
    else:
        x = np.asarray(var, dtype=float).ravel()
        names = Misc.get("active_vars")
        var_dict = {name: {"value": float(val), "active": True}
                    for name, val in zip(names, x)}

    # Clear everything in MAPDL

    mapdl.clear()


    # Create input file for Eigenvalue Analysis

    apdl_cmds = InputFun(var_dict,Misc,opti_settings)



    with mapdl.non_interactive:
        for cmd in apdl_cmds:
            cmd = cmd.strip()
            if cmd:
                mapdl.run(cmd)

    # Clear APDL

```

```
        mapdl.finish()


        # Read First eigenvalue:
        with open("Ansout/Eigenvalue1.txt") as f:
            eigenvalues = [float(line.strip()) for line in f if line.strip()]
        # Retrieve first positive eigenvalue
        alpha_crit = next(v for v in eigenvalues if v > 0)
        print(f"Critical Alpha: {alpha_crit}")

        # Open and Read Mass
        with open("Ansout/Mass_Assembly.txt","r") as f:
            Mass = [float(line.strip()) for line in f if line.strip()]

        # Return Mass as float value (evt. flyt)
        # Read per-segment mass if available
        segment_masses = []
        segment_file = "Ansout/MASS_segments.txt"
        if os.path.exists(segment_file):
            with open(segment_file, "r") as f:
                segment_masses = [float(line.strip()) for line in f if line.strip()]


        # Return Mass as float value
        return sum(Mass), segment_masses
```